



Python for Data Science (PDS)

Lab - 14

Roll No. : 122

Name : Yatharth Sanghavi

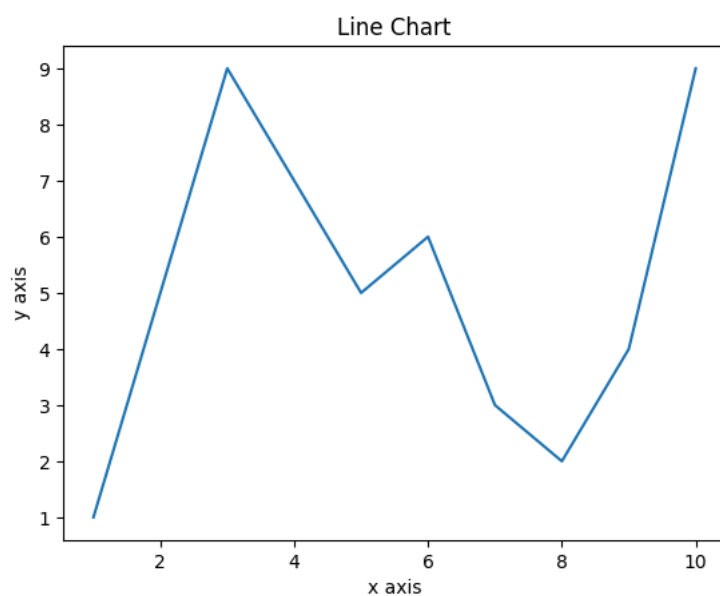
✓ Import matplotlib below

```
import matplotlib.pyplot as plt
```

✓ 01) WAP to display the line chart for given data for x and y axis.

```
x = range(1,11)  
y = [1,5,9,7,5,6,3,2,4,9]
```

```
plt.plot(x, y)  
plt.xlabel("x axis")  
plt.ylabel("y axis")  
plt.title("Line Chart")  
plt.show()
```

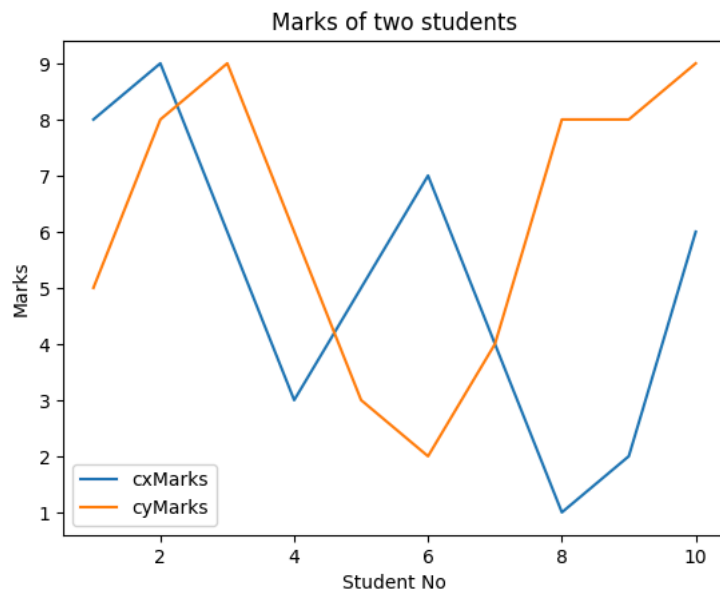


✓ 02) WAP to generate line chart for given data.

```
x = range(1,11,1)  
cxMarks= [8,9,6,3,5,7,4,1,2,6]
```

```
cyMarks= [5,8,9,6,3,2,4,8,8,9]
```

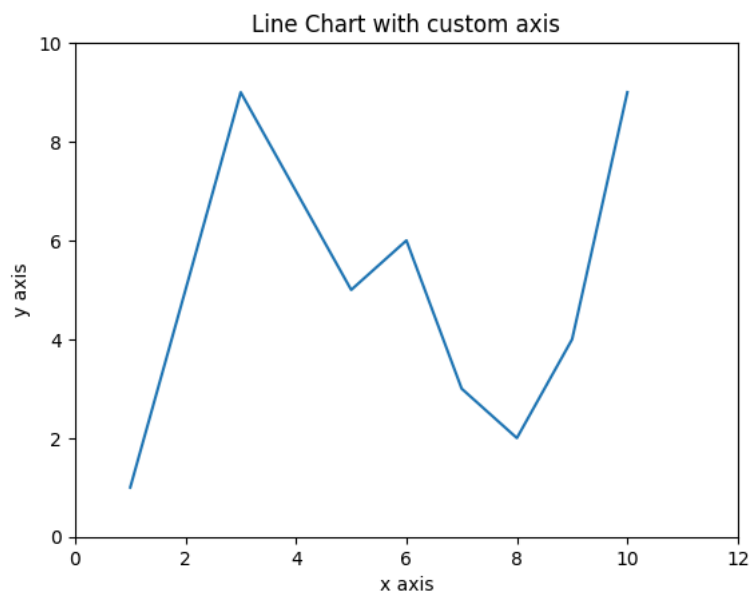
```
plt.plot(x, cxMarks, label="cxMarks")
plt.plot(x, cyMarks, label="cyMarks")
plt.xlabel("Student No")
plt.ylabel("Marks")
plt.title("Marks of two students")
plt.legend()
plt.show()
```



- 03) WAP to generate line chart for given data, but create your custom axis.

```
x = range(1,11)
y = [1,5,9,7,5,6,3,2,4,9]
```

```
plt.plot(x, y)
plt.axis([0, 12, 0, 10])
plt.xticks(range(0, 13, 2))
plt.yticks(range(0, 11, 2))
plt.xlabel("x axis")
plt.ylabel("y axis")
plt.title("Line Chart with custom axis")
plt.show()
```

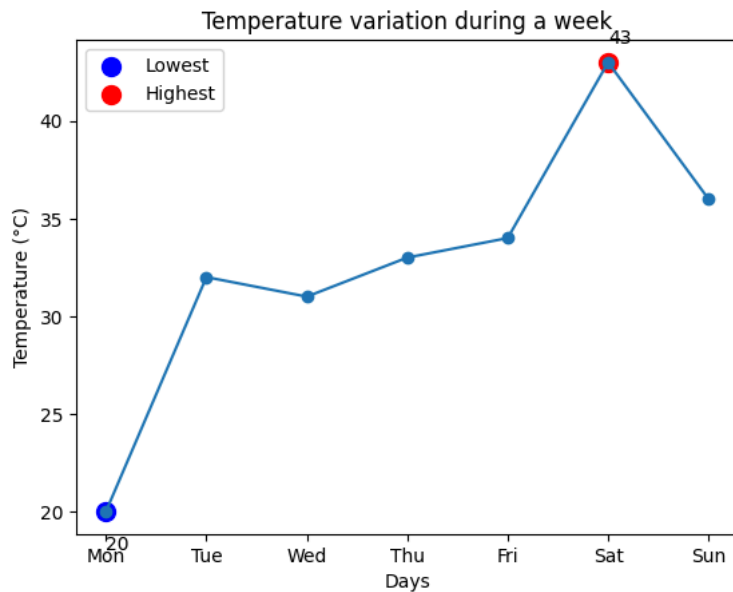


- 04) WAP to plot a line chart showing temperature variation during a week. Mark the lowest and highest temperature.

```
Days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
Temperature = [20, 32, 31, 33, 34, 43, 36]
```

```
min_temp = min(Temperature)
max_temp = max(Temperature)
min_day = Days[Temperature.index(min_temp)]
max_day = Days[Temperature.index(max_temp)]

plt.plot(Days, Temperature, marker="o")
plt.scatter(min_day, min_temp, color="blue", s=100, label="Lowest")
plt.scatter(max_day, max_temp, color="red", s=100, label="Highest")
plt.text(min_day, min_temp - 2, str(min_temp))
plt.text(max_day, max_temp + 1, str(max_temp))
plt.xlabel("Days")
plt.ylabel("Temperature (°C)")
plt.title("Temperature variation during a week")
plt.legend()
plt.show()
```



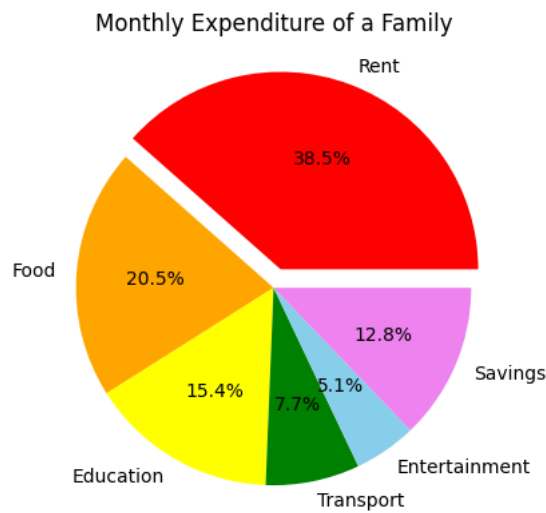
- ✓ 05) WAP to create a detailed pie chart representing the monthly expenditure distribution of a family.

The chart should display labels, percentage values, title, different colors for each slice, and explode one slice to highlight the highest expense.

```
Categories = ["Rent", "Food", "Education", "Transport", "Entertainment", "Savings"]
Amount = [15000, 8000, 6000, 3000, 2000, 5000]
```

```
colors = ["red", "orange", "yellow", "green", "skyblue", "violet"]
explode = [0.1, 0, 0, 0, 0, 0]

plt.pie(Amount, labels=Categories, colors=colors, explode=explode, autopct="%1.1f%%")
plt.title("Monthly Expenditure of a Family")
plt.show()
```

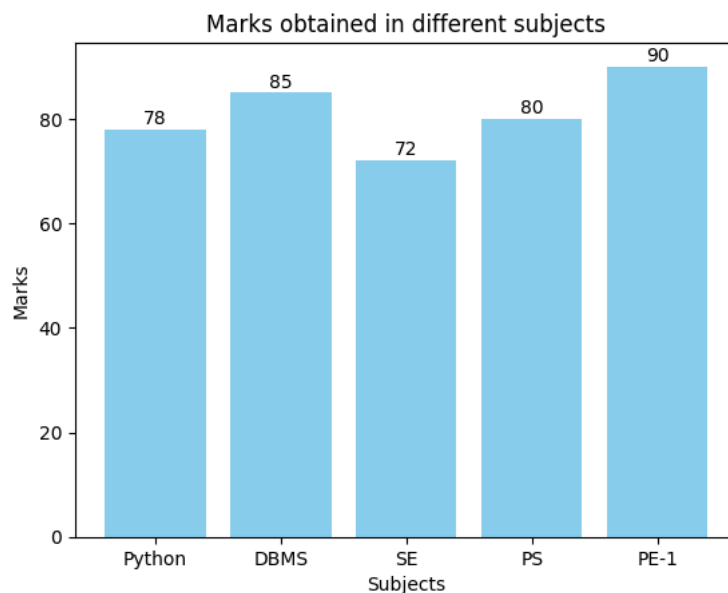


- ✓ 06) WAP to create bar chart showing the marks obtained by students in different subjects.

The program should also display the marks on the top of the respective bar.

```
Subjects = ["Python", "DBMS", "SE", "PS", "PE-1"]
Marks = [78, 85, 72, 80, 90]
```

```
plt.bar(Subjects, Marks, color="skyblue")
for i in range(len(Subjects)):
    plt.text(i, Marks[i] + 1, str(Marks[i]), ha="center")
plt.xlabel("Subjects")
plt.ylabel("Marks")
plt.title("Marks obtained in different subjects")
plt.show()
```



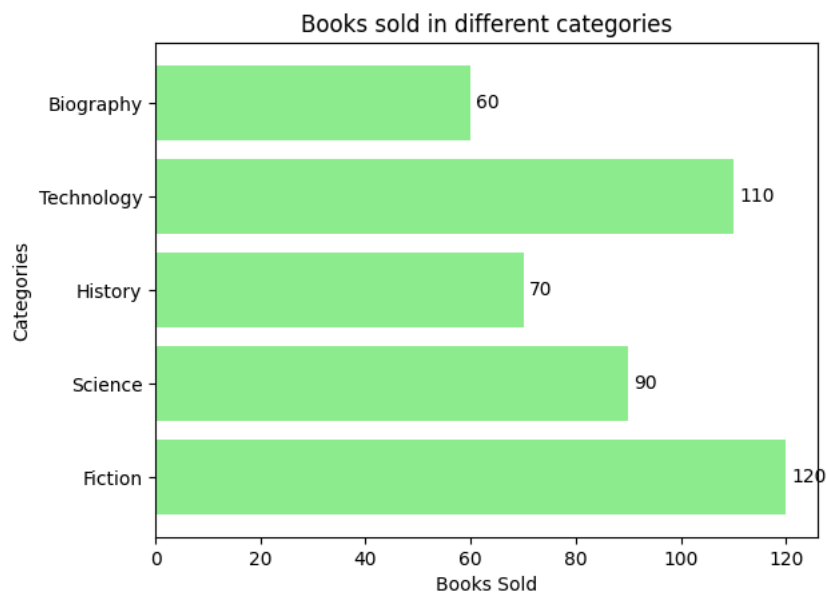
- ✓ 07) WAP to create a horizontal bar chart representing the number of books sold in different categories.

The program should also display the number of books sold at the end of the respective bar.

```
Categories = ["Fiction", "Science", "History", "Technology", "Biography"]
BooksSold = [120, 90, 70, 110, 60]
```

```
plt.barh(Categories, BooksSold, color="lightgreen")
for i in range(len(Categories)):
    plt.text(BooksSold[i] + 1, i, str(BooksSold[i]), va="center")
plt.xlabel("Books Sold")
plt.ylabel("Categories")
```

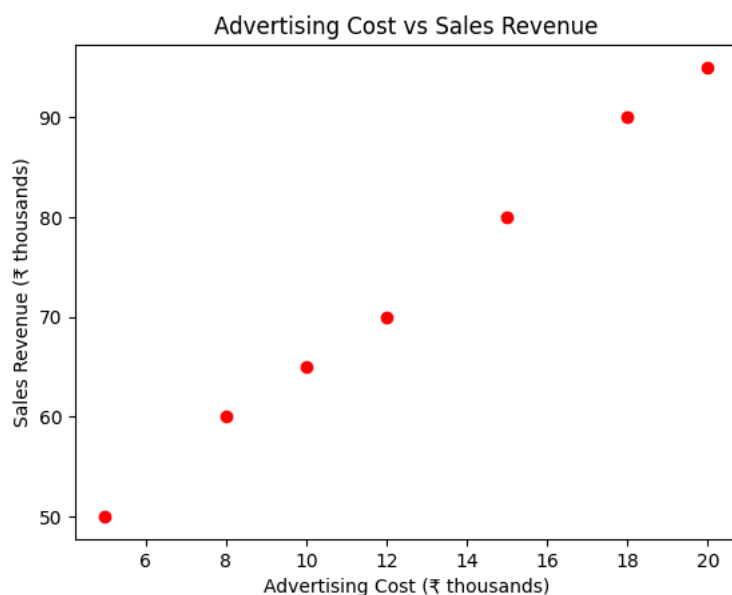
```
plt.title("Books sold in different categories")
plt.show()
```



08) WAP to create a scatter plot representing the relationship between advertising expenditure and sales revenue of a company.

```
AdvertisingCost = [5, 8, 10, 12, 15, 18, 20]
SalesRevenue = [50, 60, 65, 70, 80, 90, 95]
```

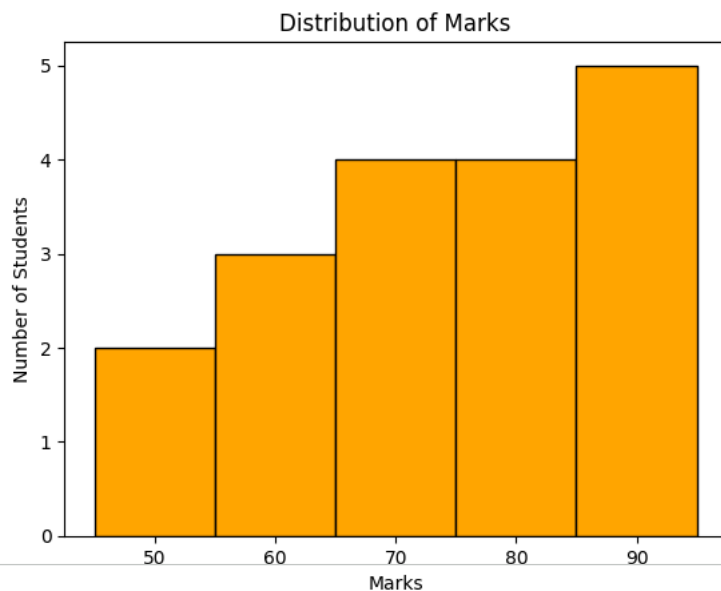
```
plt.scatter(AdvertisingCost, SalesRevenue, color="red")
plt.xlabel("Advertising Cost (₹ thousands)")
plt.ylabel("Sales Revenue (₹ thousands)")
plt.title("Advertising Cost vs Sales Revenue")
plt.show()
```



09) WAP to create a histogram representing the distribution of marks obtained by students in a class.

```
Marks = [45, 50, 55, 60, 62, 65, 68, 70, 72, 75, 78, 80, 82, 85, 88, 90, 92, 95]
```

```
plt.hist(Marks, bins=5, edgecolor="black", color="orange")
plt.xlabel("Marks")
plt.ylabel("Number of Students")
plt.title("Distribution of Marks")
plt.show()
```

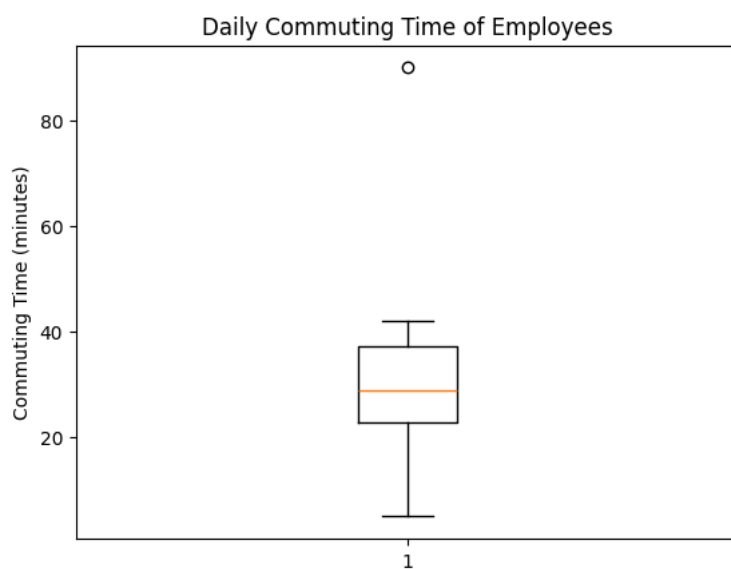


- 10) WAP to create a box plot representing the distribution of daily commuting time (in minutes) of employees and identify outliers (if any).

```
CommutingTime = [5, 18, 20, 22, 25, 27, 28, 30, 32, 35, 38, 40, 42, 90]
```

```
plt.boxplot(CommutingTime)
plt.ylabel("Commuting Time (minutes)")
plt.title("Daily Commuting Time of Employees")
plt.show()

# Outlier check using the 1.5 * IQR rule
data = sorted(CommutingTime)
n = len(data)
q1 = data[n // 4]
q3 = data[(3 * n) // 4]
iqr = q3 - q1
lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr
outliers = [v for v in data if v < lower or v > upper]
print("Outliers:", outliers)
```



```
Outliers: [90]
```